❸ Retrieves the IOptions object, which retrieves the service from DI and runs the lambda method

With the configuration in listing 31.14, when the `IOptions<CurrencyOptions>` is first injected into the minimal API endpoint, the `IOptions<CurrencyOptions>` object is built as described by the `OptionsBuilder`. First, the `"Currencies"` section of the app `IConfiguration` is bound to a new `CurrencyOptions` object. Then the `ICurrencyProvider` is retrieved from DI and passed to the `Configure<TDep>` lambda, along with the options object. Finally, the `IOptions` object is injected into the endpoint.

WARNING You must inject only singleton services using `Configure<TDeps>` methods. If you try to inject a scoped service, such as a `DbContext`, you will get an error in development warning you about a captive dependency. I describe how to work around this on my blog at http://mng.bz/7Dve.

The `OptionsBuilder<T>` is a convenient way to configure your `IOptions` objects using dependencies, but you can use an alternative approach: implementing the `IConfigureOptions<T>` interface. You implement this interface in a configuration class and use it to configure the `IOptions<T>` object in any way you need, as shown in the following listing. This class can use DI, so you can easily use any other required services.

Listing 31.15 Implementing `IConfigureOptions<T>` to configure an options object

```
public class ConfigureCurrencyOptions : IConfigureOptions<CurrencyOptions>
{
    private readonly ICurrencyProvider _currencyProvider;                ❶
```